

lab-java-add-and-update

LAB *Mandatory

Lab | Add and Update

[Submit lab](#)

Este proyecto es la continuación de lab-java-intro-to-spring-boot-openbank-2025 donde aquí se amplían endpoints usando los verbos POST Y PATCH.

A continuación, se describen los controladores y endpoints para pacientes y empleados:

patient-controller		^
POST	/api/patient/add-new-patient	▼
PATCH	/api/patient/update/{id}	▼
GET	/api/patient	▼
GET	/api/patient/{id}	▼
GET	/api/patient/employee-off/{idEmployeeStatusOff}	▼
GET	/api/patient/by-range-date	▼
GET	/api/patient/admit-employee/{employeeId}	▼
employee-controller		^
POST	/api/employee/add-new-doctor	▼
PATCH	/api/employee/update-status/{id}	▼
PATCH	/api/employee/update-department/{id}	▼
GET	/api/employee	▼
GET	/api/employee/{id}	▼
GET	/api/employee/status/{status}	▼
GET	/api/employee/departement/{department}	▼

Acceso a la documentación generado con swagger a través de:

<http://localhost:8080/swagger-ui/index.html#/>

1. **Add new patient:** Create a route to add a new patient.

The screenshot displays a REST client interface with a POST request to `http://localhost:8080/api/patient/add-new-patient`. The request body is a JSON object. The response is a 201 Created status with a JSON body containing the created patient's details.

Request:

```
POST http://localhost:8080/api/patient/add-new-patient

{
  "name": "Rogelio Losas",
  "dateOfBirth": "1980-06-01",
  "admittedBy": {
    "employeeId": 356712,
    "department": "cardiology",
    "name": "Alonso Flores",
    "status": "on_call"
  }
}
```

Response: 201 Created • 102 ms • 337 B

```
{
  "patientId": 9,
  "name": "Rogelio Losas",
  "dateOfBirth": "1980-06-01",
  "admittedBy": {
    "employeeId": 356712,
    "department": "cardiology",
    "name": "Alonso Flores",
    "status": "on_call"
  }
}
```

2. **Add new doctor:** Create a route to add a new doctor.

The screenshot displays a REST client interface with a POST request to `http://localhost:8080/employee/add-new-doctor`. The request body is a JSON object with the following structure:

```
1 {  
2   "department": "PSYCHIATRIC",  
3   "name": "Anabel Fandin",  
4   "status": "ON"  
5 }
```

The response status is **201 Created**. The response body is also a JSON object, which includes an additional `employeeId` field:

```
1 {  
2   "employeeId": 761528,  
3   "department": "PSYCHIATRIC",  
4   "name": "Anabel Fandin",  
5   "status": "ON"  
6 }
```

3. **Change doctor status:** Create a route to change a doctor's status

The screenshot displays a REST client interface with a PATCH request to `http://localhost:8080/api/employee/update-status/761528`. The 'Body' tab is selected, showing a JSON payload: `{ "status": "off" }`. The 'raw' radio button is chosen for the body type. Below the request, the response is shown with a status of `200 OK`. The response body is a JSON object: `{ "employeeId": 761528, "department": "orthopaedic", "name": "Anabel Fandin", "status": "off" }`. The interface includes tabs for Params, Authorization, Headers (9), Body, Scripts, Tests, and Settings. The response section has tabs for Body, Cookies, Headers (5), and Test Results, along with a 'Visualize' button.

```
PATCH http://localhost:8080/api/employee/update-status/761528
```

Params Authorization Headers (9) **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "status": "off"
3 }
```

Body Cookies Headers (5) Test Results 200 OK • 41

{ JSON Preview Visualize

```
1 {
2   "employeeId": 761528,
3   "department": "orthopaedic",
4   "name": "Anabel Fandin",
5   "status": "off"
6 }
```

4. **Update doctor's department:** Create a route to update a doctor's department.

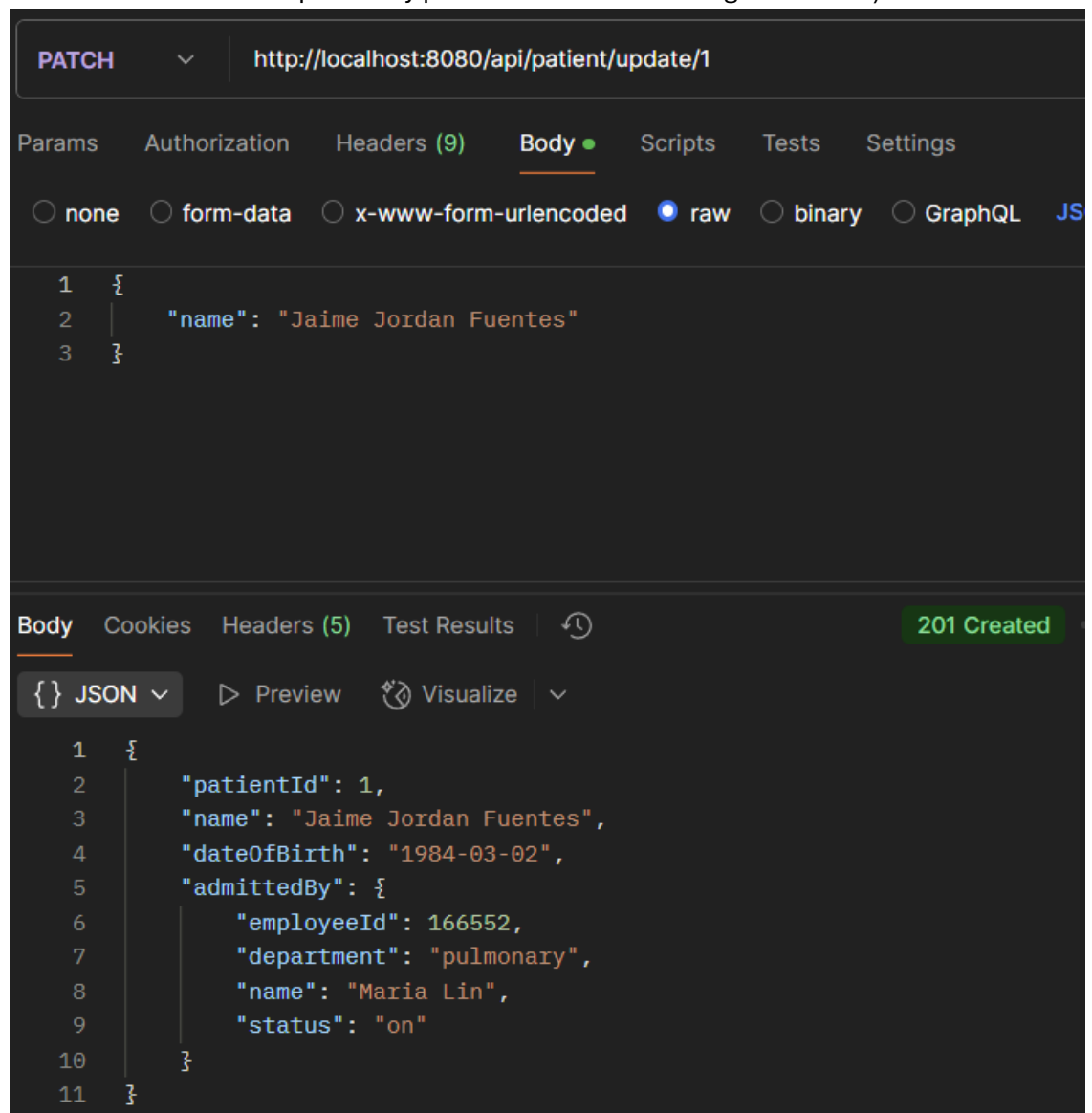
The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** `http://localhost:8080/api/employee/update-department/761528`
- Body Type:** raw (selected)
- Request Body:**

```
1 {  
2   "department": "orthopaedic"  
3 }
```
- Response Status:** 200 OK
- Response Body Type:** JSON (selected)
- Response Body:**

```
1 {  
2   "employeeId": 761528,  
3   "department": "orthopaedic",  
4   "name": "Anabel Fandin",  
5   "status": "off"  
6 }
```

5. **Update patient information:** Create a route to update patient information (the user should be able to update any patient information through this route).



Ejemplo Patch modificando sólo el nombre del paciente a través del endpoint `patient/update`

The screenshot shows a REST client interface with a PATCH request to `http://localhost:8080/api/patient/update/1`. The 'Body' tab is selected, and the request body is a JSON object. The response status is `201 Created`.

Request Body:

```
1 {
2   "admittedBy": {
3     "employeeId": 166552,
4     "department": "PULMONARY",
5     "name": "Maria Lin",
6     "status": "ON"
7   }
8 }
```

Response Body:

```
1 {
2   "patientId": 1,
3   "name": "Jaime Jordan Fuentes",
4   "dateOfBirth": "1984-03-02",
5   "admittedBy": {
6     "employeeId": 166552,
7     "department": "pulmonary",
8     "name": "Maria Lin",
9     "status": "on"
10  }
11 }
```

Ejemplo Patch modificando el doctor del paciente a través del endpoint `patient/update`. El campo ENUMERADO `department` es aceptado en mayúsculas o minúsculas gracias al uso de `@JsonCreator`

6.

- ¿Utilizó el mismo tipo de ruta para actualizar la información del paciente que para actualizar el departamento de un empleado?

No las rutas para empleados están bajo el endpoint `api/employee` y las rutas para pacientes están en el endpoint `api/patient`

- ¿Por qué eligió la estrategia seleccionada?

Se adopta esta estrategia para tener una clara diferenciación y separación de elementos intentando dar más claridad en el diseño de los controllers.

- ¿Cuál es el costo-beneficio entre usar PUT y PATCH?

Put obliga a realizar una modificación de todo un registro con lo que implica todos los valores de los campos mientras que patch permite modificar un campo específico siendo más granular.

